PROCEDRA

AI drafts the work instruction, the engineer stays accountable

Human-in-the-loop AI for review-ready drafts of industrial work instructions

Aleksandr Shuvalov

Sole developer, Procedra

Code, tests
and documentation

Industrial instructions are written by hand, depend on one author, and can't be traced to sources

01

A process engineer, supervisor or safety specialist writes the instruction outside their main workload

02

Structure and completeness depend on the individual

03

When equipment, tooling or regulations change, the document lags behind

04

There is no way to show which sources a step rests on, or who reviewed it

A plain LLM doesn't solve it: it invents settings and leaves no trace of review

A FREE-FORM CHAT

Plausible settings, tolerances and references to standards, with no source

No version, no review, no answer to “who checked this”

WHAT THE SHOP FLOOR NEEDS

Every claim marked either source-supported or needing local verification

A change log and expert sign-off before anything is used

FROM AN INTERNAL AUDIT

In an early version of Procedra, adversarial context was marked as source-supported. That led to a typed provenance model and blocking of unsafe sources.

Every claim carries provenance; the whole review cycle lives in one artifact

Claim-level provenance

Exact settings, tolerances, work permits and responsible roles are not treated as verified by default.

`validated_local`

`needs verification` `hypothesis`

Source-supported generation

A draft is built only from uploaded documents, the local index and public sources.

The model's output is schema-validated; on failure a deterministic fallback takes over.

One artifact holds the cycle

Draft → scoring → expert review → version → trial run with execution evidence → audit trail with a hash chain.

Described as a design-science artifact in a working paper (SSRN, 22 July 2026, DOI 10.2139/ssrn.7026918). Appendix A links every claim to an evidence file.

A pipeline with guardrails: sources, schema, deterministic scoring, review, audit

Request

Task, area, equipment, operation

Sources

.txt / .md / .pdf documents, local index, video

Generation

Against a Pydantic schema

Scoring

12 criteria, no external AI

Expert review

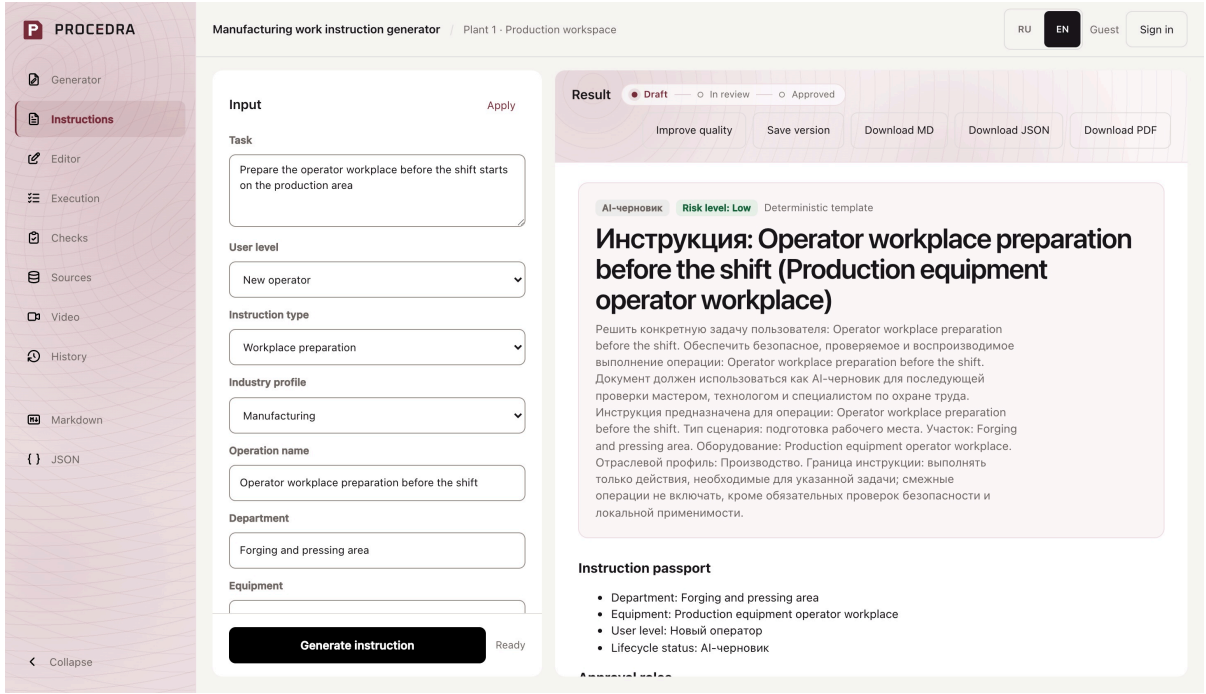
Sign-off and versions

Trial run

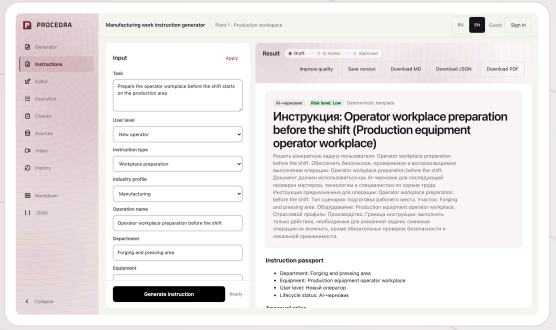
With execution evidence

Export and audit

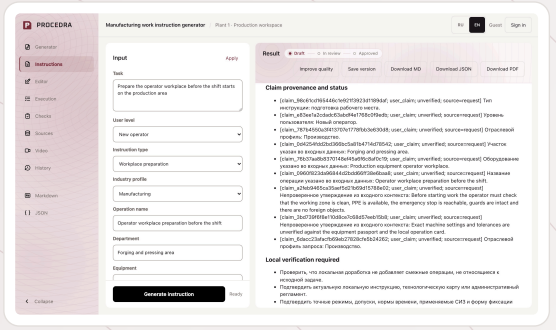
PDF / Markdown / JSON



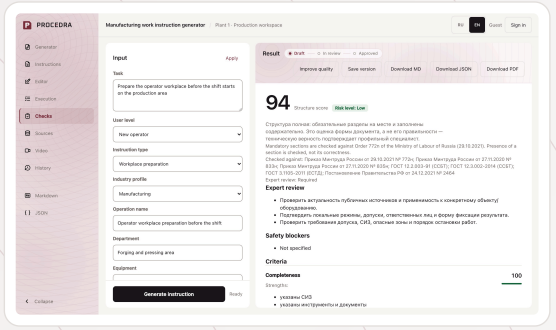
What it looks like: eight screens of the real interface



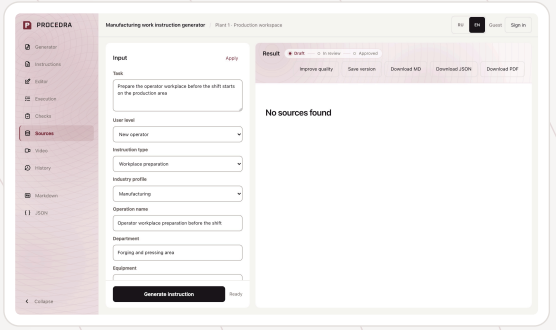
Generator



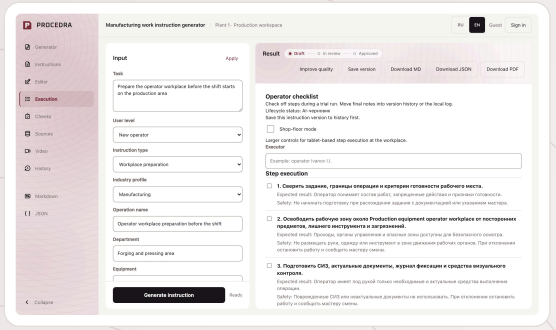
Claim provenance



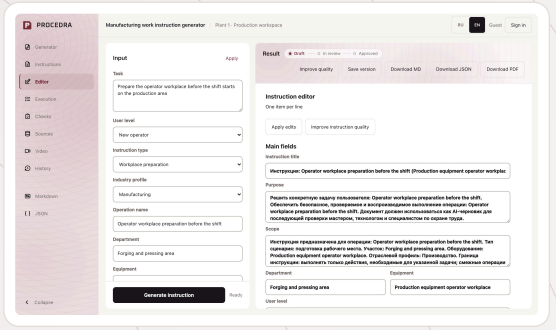
Deterministic scoring



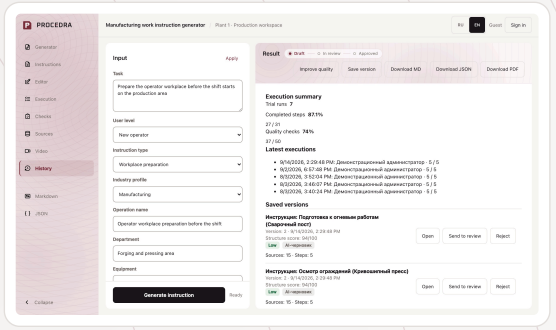
Sources



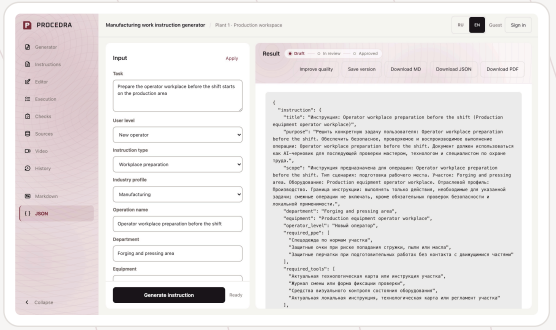
Execution checklist



Draft editor



Version history and audit



JSON export

Interface in English; generated instruction content shown in Russian.

Where it stands: public code, 459 tests, a published working paper — no field validation yet

IN PLACE

Public repository, CI, Docker image, a reproducible controlled local demo

459 tests, mypy, Ruff (local run, 2 September 2026)

15 deterministic demo scoring scenarios

A security package for a customer's security team, and an SBOM of dependencies

Working paper on SSRN, 22 July 2026

An article about building the project, published on Habr

NOT YET

A pilot on a real shop floor, and a measured effect

Validation as a safety system or proof of regulatory compliance

An internet-facing production deployment

What it deliberately does not do

01 It does not replace the process engineer, the safety specialist, or the approved local instruction

02 It does not issue work permits and does not certify compliance with standards

03 Exact settings, parameters, tolerances and responsible roles stay the plant's responsibility to verify

04 A first demo runs on synthetic or non-confidential data only

Author and links

AUTHOR

Aleksandr Shuvalov

Sole developer: architecture, code, tests, security package, paper

MSc student, South Ural State University, Computer Science and Engineering, Digital Ural Center, class of 2027

skritosss.github.io/procedra

ssrn.com/abstract=7026918 · DOI 10.2139/ssrn.7026918

[LinkedIn] · [e-mail]

LINKS

GitHub

SSRN

Habr